



Modificadores de Acesso em Herança e Slicing

Prof. Dr. Valter Vieira de Camargo

POO - 2026/1



Modificadores de Acesso

- ♦ Conceito de encapsulamento
 - encapsular é esconder o que não precisa ser acessado externamente
 - ou para proteger e controlar melhor o acesso a determinados dados/atributos
- ♦ Sendo mais claro: controlar quais elementos (atributos, métodos, classes) uma classe pode ver/acessar de outra



Modificadores de Acesso

Modificador	Acesso direto pela própria classe	Acesso direto por subclasses	Acesso direto por outras classes
private	sim	não	não
protected	sim	sim	não
public	sim	sim	sim

Nem mesmo a classe filha tem acesso direto



```
class Automovel {
```

```
private:
```

```
    string marca;  
    string modelo;
```

```
public:
```

```
    Automovel(const string& marca, const string& modelo)  
        : marca(marca), modelo(modelo) {}
```

```
class Carro : public Automovel {
```

```
private:
```

```
    int numeroDePortas;
```

```
public:
```

```
    ...
```

```
    void exibirInformacoes() const override {  
        cout << "Carro: " << getMarca() << " " << getModelo()  
            << ", Portas: " << numeroDePortas << endl;  
    }
```

Mesmo sendo uma herança, marca e modelo não podem ser acessados diretamente, mas numeroDePortas pode ...



```
class Automovel {
```

```
protected:
```

```
    string marca;  
    string modelo;
```

Protected

```
public:
```

```
    Automovel(const string& marca, const string& modelo)  
        : marca(marca), modelo(modelo) {}
```

```
class Carro : public Automovel {
```

```
private:
```

```
    int numeroDePortas;
```

```
public:
```

```
    ...
```

```
    void exibirInformacoes() const override {  
        cout << "Carro: " << marca << " " << modelo  
            << ", Portas: " << numeroDePortas << endl;  
    }
```

Agora, marca e modelo podem ser acessados diretamente, sem precisar dos gets públicos. Mas continuam inacessíveis externamente



```
class Pessoa {
```

```
    private:  
        string cpf;
```

```
    protected:  
        string nome;
```

```
    public:
```

```
        int idade;
```

```
        void setCpf(string c) { cpf = c; }  
        string getCpf() { return cpf; }
```

```
};
```

Observe que pode-se ter
atributos e métodos com
diferentes modificadores ...



Tipos de Herança

public/protected/private

- ♦ Além de poder usar os modificadores de acesso nos atributos e métodos, pode-se também estipular o “tipo” da herança
- ♦ Isto é, pode-se criar uma herança pública, ou protegida ou privada...
- ♦ Observe que até este momento do curso, sempre usamos herança pública nos exemplos:

class Assalariado : public Funcionario

Tipos de Herança

public/protected/private

Mudanças importantes !

- ♦ Em C++, pode-se escolher o tipo de Herança

Tipo de membro no Pai

Com herança pública

public

continua public na filha

protected

continua protected na filha

private

não é herdado diretamente (continua inacessível)

Com herança protegida

vira protected

continua protected

não é herdado diretamente

Com herança privada

vira private

vira private

não é herdado diretamente

🧠 Resumo mental:

Herança pública: "preserva" o acesso.

Herança protegida: "restringe um pouco" (público vira protegido).

Herança privada: "fecha tudo" (tudo vira privado).



Tipos de Herança

public/protected/private

```
class Carro : Automovel {  
private:  
    int numeroDePortas;  
  
public:  
    Carro(const string& marca, const string& modelo, int portas)  
        : Automovel(marca, modelo), numeroDePortas(portas) {}  
}
```

Cuidado !
Se não usar nenhum
modificador de acesso, o
default é “private”



Cuidado com o default

Herança privada - torna tudo privado na filha.

```
class Automovel {
```

```
private:
```

```
    string marca;  
    string modelo;
```

```
public:
```

```
    string teste;  
    .....
```

Atributo público ...

```
class Carro : Automovel {
```

```
private:
```

```
    int numeroDePortas;
```

```
public:
```

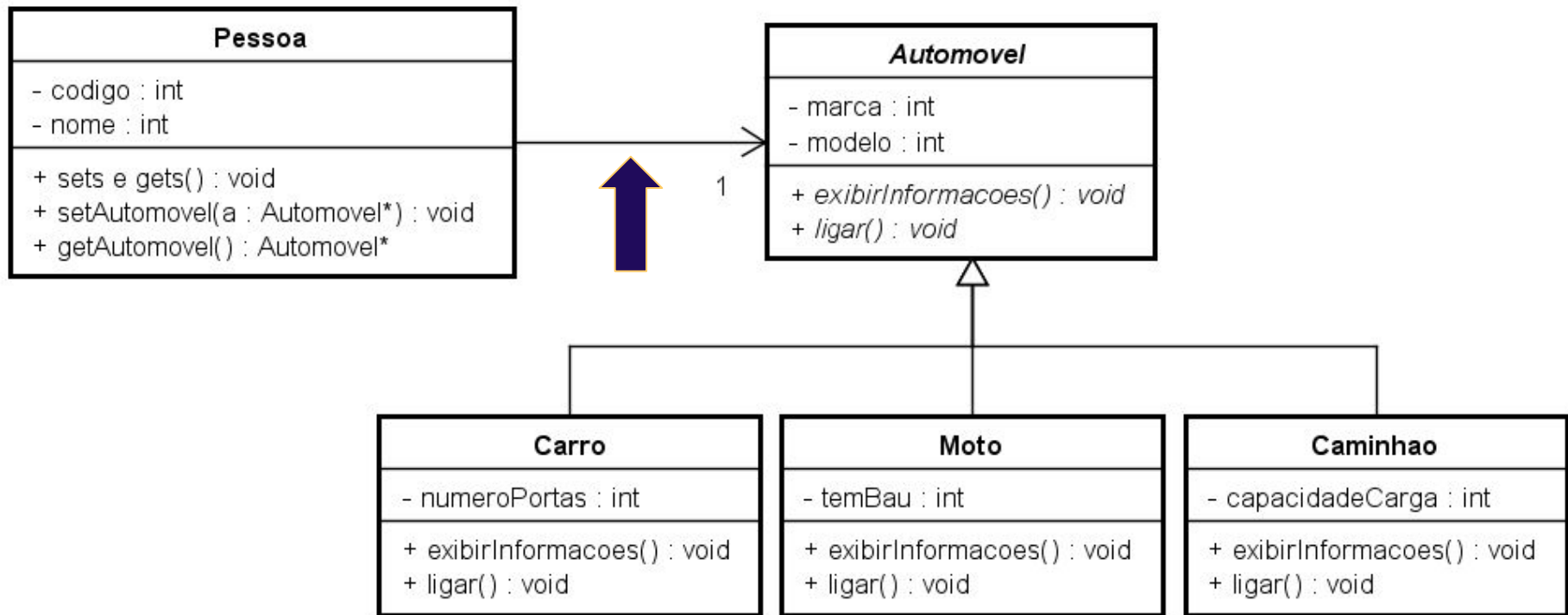
```
    void exibirInformacoes() const override {  
        cout << "Carro: " << getMarca() << " " <<  
        getModelo()  
        << ", Portas: " << numeroDePortas << endl;  
    }
```

```
    void ligar() const override {  
        cout << "teste " << teste;  
        cout << "O carro está ligado!" << endl;  
    }  
};
```

Se torna privado, mas pode ser acessado de dentro da filha ...só fora que não ... isto é, ele se torna privado para classes externas

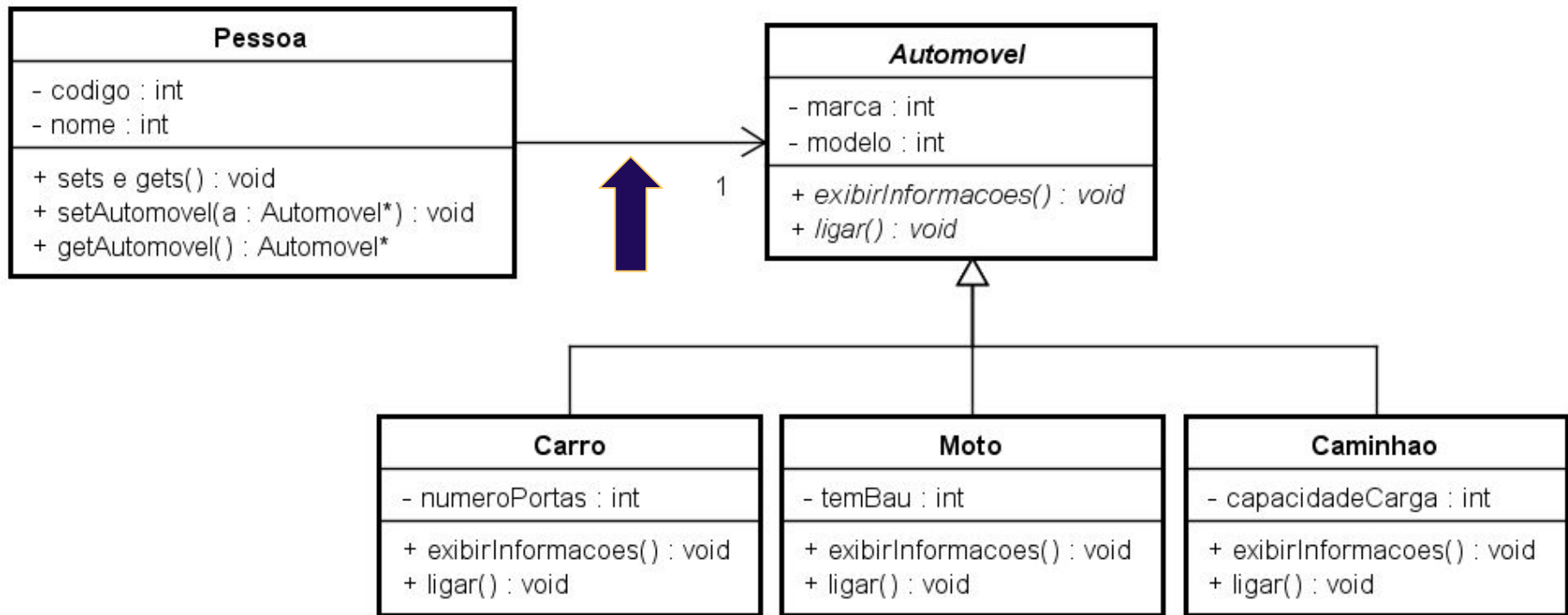


Relacionamento de classe Cliente com Hierarquia



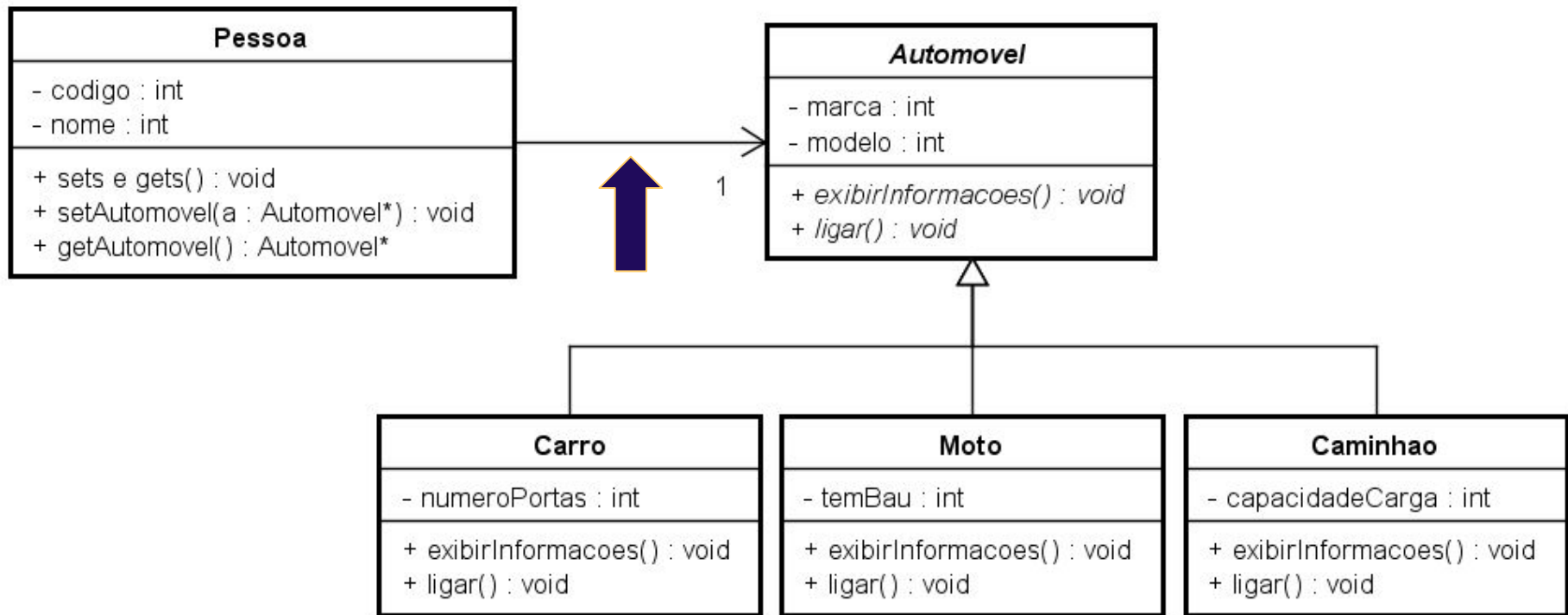
powered by Astah

Um ponto muito importante é a forma correta de se criar o relacionamento com a classe Base de uma hierarquia de herança



powered by Astah

Sabemos que a ideia é “criar um atributo em Pessoa do tipo Automovel”... mas .. como ?
atributo direto ou ponteiro ?



powered by Astah

A forma correta é ponteiro, pois é a única forma de se garantir que o polimorfismo irá funcionar adequadamente.

Relacionamento com Classe Base

```
class Pessoa {
```

```
private:
```

```
    int codigo;
```

```
    string nome;
```

```
    Automovel automovel;
```

```
public:
```

```
    //construtores...
```

```
    //sets e gets
```

ERRADO !!!!

**Não vai funcionar o
polimorfismo !**

Problema de “Slicing”

Relacionamento com Classe Base

```
class Pessoa {
```

```
private:
```

```
    int codigo;
```

```
    string nome;
```

```
    Automovel* automovel;
```

```
public:
```

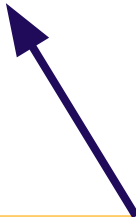
```
    //construtores...
```

```
    //sets e gets
```

CORRETO !
Ponteiro para
automóvel.

Relacionamento com Classe Base

```
class Pessoa {  
  
private:  
    int codigo;  
    string nome;  
    Automovel* automovel; // ponteiro para automóvel (classe base)  
  
public:  
    Pessoa(int codigo, const string& nome)  
        : codigo(codigo), nome(nome), automovel(nullptr) {}  
  
    Pessoa(string n, Automovel* a)  
        : codigo(codigo), nome(n), automovel(a) { }  
  
    void setAutomovel(Automovel* a) {  
        automovel = a;  
    }  
  
    Automovel* getAutomovel() const {  
        return automovel;  
    }  
};  
\\continua no próximo slide
```

- 
- Inicialize o ponteiro automovel com um valor nulo
 - melhor do que não inicializar ele com nada ... pode levar a um valor indefinido

Relacionamento com Classe Base

```
void exibirInformacoes() const {
```

```
    cout << "Pessoa: " << codigo << " - " << nome << endl;
```

```
    if (automovel) {
```

Se “automóvel” for diferente de nulo ...

```
        cout << "Automóvel da pessoa:" << endl;
```

```
        automovel->exibirInformacoes(); // polimorfismo aqui!
```

```
    } else {
```

```
        cout << "Nenhum automóvel associado." << endl;
```

```
    }
```

O método `exibirInformações()` que será executado depende de qual é o objetivo automóvel no momento da execução...

```
~Pessoa() {
```

```
    delete automovel;
```

```
    cout << "Pessoa destruida\n";
```

```
}
```

```
}
```

Precisa declarar o destrutor por causa do ponteiro



```
class Automovel {  
protected:  
    string modelo;  
  
public:  
    Automovel(string m) : modelo(m) {  
        cout << "Automovel criado\n";  
    }  
  
    virtual void exibirInformacoes() {  
        cout << "Modelo: " << modelo << endl;  
    }  
  
    virtual ~Automovel() {  
        cout << "Destruindo Automovel\n";  
    }  
};
```



```
class Carro : public Automovel {
public:
    Carro(string m) : Automovel(m) {
        cout << "Carro criado\n";
    }

    void exibirInfo() override {
        cout << "Carro -> Modelo: " << modelo <<
endl;
    }

    ~Carro() override {
        cout << "Destruindo Carro\n";
    }
};
```



No Main...

```
int main() {
```

```
    Automovel* c1 = new Carro("Toyota", "Corolla", 4);  
    Automovel* m1 = new Moto("Honda", "CG 160", true);
```

```
    c1->exibirInformacoes();  
    m1->exibirInformacoes();
```

```
    cout << endl;
```

```
    Pessoa* valter = new Pessoa(1,"Valter", c1);  
    Pessoa* arthur = new Pessoa(2,"Arthur", m1);
```

```
    valter->exibirInformacoes();  
    arthur->exibirInformacoes();
```

```
    cout << endl;
```

```
    return 0;
```

```
}
```



Entendendo o problema do Slicing

- ◆ Ocorre quando um objeto da classe derivada é copiado para a classe base “por valor”
- ◆ o problema é que o pedaço da classe derivada é cortado/perdido

Entendendo o problema do Slicing

```
class Animal {  
protected:  
string nome;  
int idade;
```

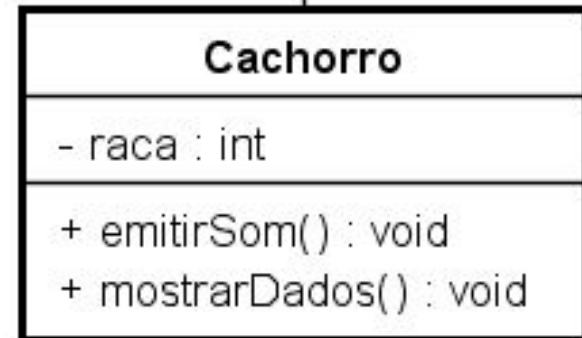
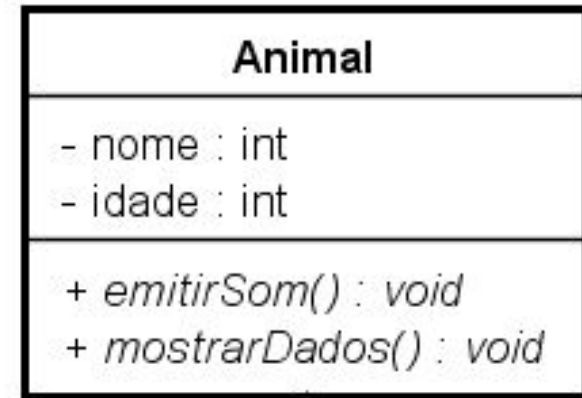
```
public:
```

```
Animal(string n, int i) : nome(n), idade(i) { }
```

```
virtual void emitirSom() {  
    cout << "Som generico" << endl;  
}
```

```
virtual void mostrarDados() {  
    cout << "Nome: " << nome << endl;  
    cout << "Idade: " << idade << endl;  
}
```

```
virtual ~Animal() = default;  
};
```



Entendendo o problema do Slicing

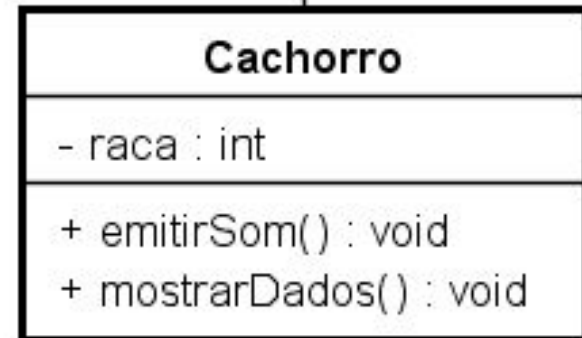
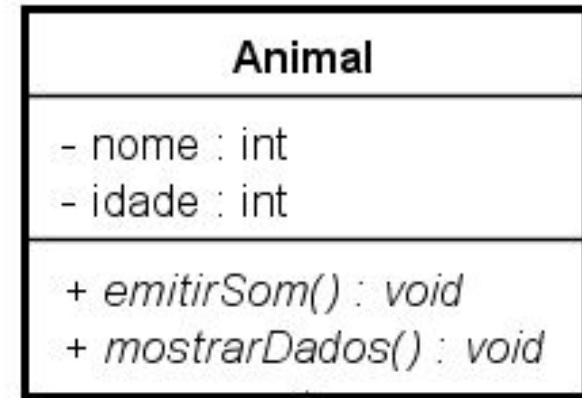
```
class Cachorro : public Animal {
```

```
private:  
    string raca;
```

```
public:  
    Cachorro(string n, int i, string r): Animal(n, i),  
    raca(r) { }
```

```
void emitirSom() override {  
    cout << "Au Au!" << endl;  
}
```

```
void mostrarDados() override {  
    cout << "=== Cachorro ===" << endl;  
    cout << "Nome: " << nome << endl;  
    cout << "Idade: " << idade << endl;  
    cout << "Raca: " << raca << endl;  
};
```



Entendendo o problema do Slicing

```
int main() {
```

```
    Cachorro c( "Rex", 5, "Labrador" );
```

```
    c.mostrarDados();
```

```
    c.emitirSom();
```

```
    Animal a = c;    //SLICING ACONTECENDO AQUI
```

```
    a.mostrarDados(); //SÓ DADOS DO ANIMAL
```

```
    a.emitirSom();    //SÓ O SOM DO ANIMAL
```

```
    return 0;
```

```
}
```



Entendendo o problema do Slicing

```
int main() {
```

```
    Cachorro c( "Rex", 5, "Labrador" );
```

```
    c.mostrarDados();
```

```
    c.emitirSom();
```

```
    Animal a = c;    //SLICING ACONTECENDO AQUI
```

// aqui o compilador cria um novo objeto *a* e copia para dentro dele só a parte que cabe lá dentro, i.e., só a parte de animal. Perde tudo (corta/slice) que for específico de cachorro.

```
    a.mostrarDados(); //SÓ DADOS DO ANIMAL
```

```
    a.emitirSom();    //SÓ O SOM DO ANIMAL
```

```
    return 0;
```

```
}
```



Entendendo o problema do Slicing

```
int main() {
```

```
    Cachorro c( "Rex", 5, "Labrador" );
```

```
    c.mostrarDados();
```

```
    c.emitirSom();
```

```
    Animal* a = &c; //PROBLEMA CORRIGIDO
```

```
    a.mostrarDados(); //imprime todos os dados
```

```
    a.emitirSom(); //som de cachorro
```

```
    return 0;
```

```
}
```





FIM

Cuidado com isso no projeto de
vocês !